Java development 2.0: Cloud-based messaging with Amazon SQS

Pay as you go, with Amazon's message-queuing system

Skill Level: Intermediate

Andrew Glover (ajglover@gmail.com)
Author and developer
Beacon50

22 Mar 2011

Amazon Simple Queue Service (SQS) borrows what it needs from message-oriented middleware (MOM) but doesn't lock you in to any one implementation language or framework. Learn how to use Amazon SQS to alleviate the burden of installing and maintaining a message-queuing system, while leveraging the pay-as-you-go scalability of AWS.

About this series
The Java development landscape has changed radically since Java™ technology first emerged. Thanks to mature open source frameworks and reliable for-rent deployment infrastructures, it's now possible to assemble, test, run, and maintain Java applications quickly and inexpensively. In this series, Andrew Glover explores the spectrum of technologies and tools that make this new Java development paradigm possible.

Messaging queues are common across a range of software architectures and domains, including financial systems, healthcare, and the travel industry. Yet message-oriented middleware (MOM) — the dominant messaging paradigm for distributed systems — requires a queuing system to be especially installed and maintained. This month, I introduce a cloud-based alternative to such labor-intensive messaging: Amazon's Simple Queue Service (SQS).

Just as it often makes sense to host a web application on Google App Engine or

Amazon Elastic Beanstalk (see Resources), so does it make sense to leverage a cloud messaging system. Either way, you are able to spend more time writing the application, rather than installing and maintaining its underlying infrastructure.

In this article, you'll learn how Amazon SQS elevates the burden of installing and maintaining a message-queuing system. You'll also have the opportunity to practice creating SQS message queues, then dropping and retrieving messages on them. Finally, I'll show you what happens when I add messaging to Magnus, the mobile web application I used for last month's introduction to Amazon Elastic Beanstalk.

Who's calling? It's MOM.

Message-oriented middleware, or MOM, is a term that describes loosely coupled systems that communicate via message queues. Rather than the components of a system being tightly coupled (via compile-time dependencies, for example) they are distributed across a network. This distributed effect, in which *message queues* are the medium for communication, enables messaging systems to scale.

Traditionally, architects have decided which components will communicate with one another in a message-oriented system. While all communication happens via message passing, the message itself is often in a generic cross-platform format. Messages could be simple strings or even documents encoded using XML or JSON.

Because a MOM architecture decouples components and enables cross-platform communication between them, individual units can be heterogenous. That is, components in a distributed architecture can be written in different languages, such as in the Java language, C#, and Ruby. Components can also exist on different platforms, like UNIX® and Windows®. What's more, MOMs make system integration easier. As middleware, MOMs can connect legacy systems as well as newer ones. This is because the API between components is simply a message, which can be anything from an XML document, to a serialized object, to a simple `String`.

GAE is your MOM!

Message queues in a MOM system are the plumbing of the web: they connect various system components in order to enable messages to flow freely between them. As it turns out, GAE is an excellent example of a message-oriented middleware system.

Like any good MOM, Google App Engine uses message queues to decouple system processes. Specifically, GAE queues make it possible to unload long-running processes from web requests. Using GAE, you dump URLs that point to servlets or JSPs onto message queues, which are then picked up and processed by GAE services. Servlets are invoked asynchronously in relation to the main logical sequence of a web application. (See Resources to learn more about GAE.)

Queueing longer running processes in order to manage the duration of main processes isn't just a GAE thing, however. This MOM-like feature is offered with other PaaS implementations such as Heroku. With Amazon SQS, however, you can do it easily in *any* web application, regardless of platform.

Introducing Amazon SQS

Amazon SQS offers a number of features that should be familiar if you've used message queues in JMS.

> **Amazon SQS is not JMS**
> Message queues on the Java platform are nothing new, as exemplified by the JMS specification. JMS is over a decade old and includes an impressive list of implementations, including RabbitMQ, Apache's ActiveMQ, and even IBM's Websphere® MQ. The Amazon SQS API doesn't implement any JMS interfaces, however. In fact, it's arguably more simple than JMS and much easier to get up and running.

Amazon SQS:

- Permits multiple processes to both read and write from the same queue. It also locks messages during processing, which ensures that a message will only be processed by one reader, even if multiple processes are reading from a single queue.
- Leverages Amazon's massively redundant architecture to offer extremely high availability in the face of concurrent access. It also guarantees delivery of a message (at least once).
- Requires that you only pay for what you use. For Amazon SQS, that means you pay $0.000001 per message. AWS currently offers a free tier, in which the first 100,000 messages per month are *gratis*. Keep in mind that there are bandwidth charges priced by the gigabyte, which is common to all AWS products.

Getting started with SQS is just as simple as everything else in AWS. If you don't already have an AWS account, first create one. Next, enable Amazon SQS. Finally, use the AWS interface Java SDK to publish and read cloud-based messages! (More about actually *writing* them below.)

Writing SQS messages

In keeping with the Amazon SQS name, the logic behind reading and writing to a queue is simplicity itself. First, establish a connection to AWS using a valid access

key and secret, as shown in Listing 1:

Listing 1. Establishing a connection to AWS

```
AmazonSQS sqs = new AmazonSQSClient(new BasicAWSCredentials(AWS_KEY, AWS_SECRET));
```

Next, you need a queue. In the AWS API, the call to `createQueue`, shown in Listing 2, doesn't necessarily create a new queue every time. If the queue already exists, its handle is returned. In SQS, queues are just URLs; consequently, a queue handle is also just a URL. Note that in the AWS SDK API, the `Queue` URL is a `String` type and not the Java `URL` type.

Listing 2. Obtaining a handle to a Queue

```
String url = sqs.createQueue(new CreateQueueRequest("a_queue")).getQueueUrl();
```

Once you have a queue, you can write a message to it. SQS's message format is similar to SimpleDB's (see Resources), in that messages are `Strings`. Remember, though, that a `String` can easily be structured, and thus easily parsed, by making its format valid JSON or XML.

Listing 3. Sending messages via SQS

```
sqs.sendMessage(new SendMessageRequest(url, "It's a wonderful life!"));
```

SQS keeps it simple

Keep in mind that Amazon SQS is first and foremost simple, which means it lacks some extras you may be used to. For instance, SQS doesn't do proactive notifications, so readers of an SQS queue must periodically poll to see if it contains new messages. While not terrible, this does add to application overhead, which might not be acceptable in some cases. Amazon Simple Notification Service (SNS) resolves this problem, but that's a subject for another article.

Message lengths are bounded. By default, a message can't exceed 8KB. If you need to use messages of greater length, you can always chop them up, identifying the individual pieces with sequence IDs. The messages can then be reassembled on the receiving side.

That's it — it took just those three lines of code to place a message on an SQS queue.

About the AWS SDK

You might be noting a familiar pattern in the AWS SDK, especially if you've read my

introduction to SimpleDB (see Resources). Because everything in AWS is a web service, all communication happens over HTTP. Consequently, the API mimics logical requests via `Request`-like objects, such as `SendMessageRequest` or `CreateQueueRequest`. In both cases, the names describe the object's intent.

Something else to note is that messages placed on SQS are durable: they are there until you remove them. (Messages do eventually disappear if you don't remove them; the default value for auto-expiration is four days.) Amazon SQS employs a simple locking strategy when messages are fetched for reading — for a read event, the message won't be available to other concurrent reading processes for a period of time, known as the message's *visibility timeout*. This value is set to 30 seconds by default, although you are free to alter the duration as you see fit.

The durableness of messages living in Amazon's infrastructure is reassuring. Like SimpleDB, and even S3, components in the AWS world are massively redundant. If your reader process (or processes) terminate unexpectedly during message processing, there's a good chance that the message will still be around. What's more, if some asset in the AWS network also decides to kick the bucket, you can bet that your mission-critical messages won't be lost — they'll still exist on any number of other machines. Lastly, as is the case with all other AWS products, you can set the physical location of your message infrastructure by region: U.S., E.U., and so on.

Reading SQS messages

Writing a message to an SQS queue takes three lines of code. Reading a message only takes a few more. In fact, the first two lines are identical, given that you need a connection to AWS and a handle to the same queue. Amazon SQS doesn't offer any call-back functionality or proactive notification of message arrivals. You must poll an SQS queue periodically to see if it has anything to deliver. Consequently, reading an SQS queue requires those few additional lines of code.

There's a slight caveat to implementing a polling strategy: you must check to ensure that you actually have received a valid message before trying to process one. If you don't, you'll surely end up seeing the nefarious `NullPointerException`.

For example, assuming I've got a valid connection to AWS and a handle to a queue containing messages, I can retrieve messages as shown in Listing 4:

Listing 4. Receiving messages via SQS

```
while (true) {
 List<Message> msgs = sqs.receiveMessage(
    new ReceiveMessageRequest(url).withMaxNumberOfMessages(1)).getMessages();

 if (msgs.size() > 0) {
  Message message = msgs.get(0);
  System.out.println("The message is " + message.getBody());
  sqs.deleteMessage(new DeleteMessageRequest(url, message.getReceiptHandle()));
```

```
    } else {
      System.out.println("nothing found, trying again in 30 seconds");
      Thread.sleep(3000);
    }
  }
```

In Listing 4, the reference to `sqs` is an `AmazonSQS` type as seen in Listing 1. This object provides a `receiveMessage` method which accepts a `ReceiveMessageRequest`. `ReceiveMessageRequests` can be configured to request a set number of messages in a queue. In my case, I've configured it to simply grab one message at a time. Regardless of how many messages I request, the `receiveMessage` method returns a `List` of `Message` types.

Implementing a polling strategy

As I previously mentioned, SQS reading is done polling-style; what's more, the `receiveMessage` method is non-blocking. Consequently, I have to check that the corresponding `List` (`msgs`) actually contains anything. If nothing was retrieved from a queue, the call to `getMessages` on the `ReceiveMessageRequest` will return an empty `List`, rather than `null`.

Provided I've retrieved a valid message, I can obtain its payload or body via the `getBody` call. Keep in mind that once you have a handle to a valid message, SQS locks it. By default I have 30 seconds to do something with the message. I must delete the message if I wish to permanently remove it from processing. Thus, I issue a `deleteMessage` call, which takes a `DeleteMessageRequest`.

A `Message` instance is distinguished by its *receipt handle*, like `id`. The handle isn't related directly to the message but more to the *event* that it is being read. A message that was read more than once (such as if it wasn't deleted, or if a reading process failed) could have multiple, yet differing, receipt handles. As a result, when you wish to delete a message, you must provide its receipt handle via the `getReceiptHandle` call.

Rather than continuously checking to see if my queue has a message, I provide a sleep function that waits 30 seconds in the event that no message was retrieved. Obviously in some cases, sleeping may not be a good idea, or a longer pause duration could be in order.

With those few lines of code, I've pretty much covered Amazon SQS. While the AWS SDK provides a number of other functions and features, the code so far is all you need to read and write messages to SQS queues.

Now let's see what happens when we actually use it.

Magnus meets Amazon SQS

Last month, I created a simple mobile web application called Magnus, which I used to demonstrate some of the features of Amazon Elastic Beanstalk (see Resources). Magnus has a nifty ability to store location information received from the mobile devices of account holders — just the sort of information that many people want to provide, and that many others want to consume.

Capturing someone's whereabouts is well and good, but what people really love are graphs (that and shiny buttons with rounded corners). Graphing and analytics can be expensive from a processing prospective when you've got tons of data to move. (Hadoop, anyone?) The tried-and-true technique of *extract, transform, and load*, or ETL, is one way to manage this. ETL is a rather large term that encompasses a lot of things. (People build careers and companies build businesses around this acronym!) In this case, ETL simply implies that I'm going to analyze some MongoDB data and create new documents based on that data.

ETL with Amazon SQS

When it comes to data analytics, there are myriad possibilities for what we can ask of data, and for the answers we can provide. The Magnus web app takes on a rather small slice of this potential: it pulls and presents data related to geographic coordinates, times, and user accounts. Technically, Magnus is interested in location latitude and longitude, the user account ID, timestamps, and the relationships between these particular data.

Magnus could give a graphical representation of this data showing user accounts by geographic area (perhaps a map with markers locating an account holder at any given time). Or it could show how an account holder/user moved over a given area (another map). Providing this sort of information involves an ETL-like process that happens offline. Providing the data in realtime, as it was generated, could be too expensive from a processing standpoint. So think of these analytics as *near-realtime*.

In order to use Amazon SQS in Magnus, I need to do some preliminary setup. First, I need a way to obtain AWS credentials. I'm fond of Play (see Resources), so I'll be using it as my application development framework. To get the credentials, I can use Play's `application.conf` file, a properties file that is read automatically.

Listing 5. Adding AWS configuration data to Play's application.conf

```
#AWS configuration
aws_access_key_id=1S.........MR2
aws_secret_access_key=S3.........ZM
```

Once the properties have been defined, I can easily obtain them via a call to Play's `Play` object, as shown in Listing 6:

Listing 6. Obtaining AWS info in Play

```
public class Application extends Controller {

 private static final String AWS_KEY =
    Play.configuration.get("aws_access_key_id").toString();
 private static final String AWS_SECRET =
    Play.configuration.get("aws_secret_access_key").toString();

//....
}
```

With that plumbing defined, I can get down to business. The code in Listing 7 is similar to a snippet I used in my introduction last month to Amazon Elastic Beanstalk. In this case, I've simply updated the `saveLocation` with some code to place a simple JSON document onto a queue named "`locations_queue`". The JSON basically looks like this: `{"id":"4d6baeb52a54f1000001"}`. The ID of the saved location is provided for the recipient of the message to look up and analyze.

Listing 7. A saveLocation method to place messages on SQS

```
public static void saveLocation(String id, JsonObject body) throws Exception {
 String eventname = body.getAsJsonPrimitive("name").getAsString();
 double latitude = body.getAsJsonPrimitive("latitude").getAsDouble();
 double longitude = body.getAsJsonPrimitive("longitude").getAsDouble();
 String when = body.getAsJsonPrimitive("timestamp").getAsString();

 SimpleDateFormat formatter =
   new SimpleDateFormat("dd-MM-yyyy HH:mm");
 Date dt = formatter.parse(when);

 ObjectId oid = new Location(id, dt, latitude, longitude).save();

 AmazonSQS sqs = new AmazonSQSClient(new BasicAWSCredentials(AWS_KEY, AWS_SECRET));

 Map mp = new HashMap<String, String>();
 mp.put("id", oid.toString());

 String url = sqs.createQueue(new CreateQueueRequest("locations_queue")).getQueueUrl();
 sqs.sendMessage(new SendMessageRequest(url, new Gson().toJson(mp)));

 renderJSON(getSuccessMessage());
}
```

A date with Ruby?

Now that messages are being placed on an SQS queue, I need to pop them off of the queue and do some processing. If you recall, one of the advantages of a MOM is that it permits heterogeneous architecture. To that end, the SQS reader side of the house could be written in a language other than Java code, and could even run on another platform!

Because I could basically do my analytics processing in anything I like, I'm going to do it in Ruby — to win some street cred with the cool kids.

In Listing 8, I've enlisted the help of the `right_aws` Ruby gem to assist me in working with SQS. In many ways, you can think of a *gem* as a jar file. The

`right_aws` library is much like Amazon's SDK for Java, albeit less verbose and a lot more straightforward to work with.

Listing 8. Creating a connection and queue in Ruby for SQS

```
require "right_aws"
#...
sqs  = RightAws::SqsGen2.new(aws_access_key_id, aws_secret_access_key)
queue = sqs.queue('locations_queue')
```

As you can see, the two lines of relevant code from Listing 8 establish a connection to AWS and grab a handle to my queue named `'locations_queue'`.

Next, I put a polling mechanism in place, as shown in Listing 9. The reference to `@queue` is the same `queue` variable from Listing 8. In this case, however, it has been defined as a part of a class. So in Listing 9, I'm directly referring to an instance variable with Ruby's @ syntax.

Listing 9. Processing messages from SQS

```
def process_messages()
  while true
    msg = @queue.pop
    if !msg.nil?
      handle_message(msg) # impl of which does neat stuff
          msg.delete
    else
      sleep 10
    end
  end
end
```

After I pass the message off to the `handle_message` method, I can delete it. If no message is found, the main thread sleeps for 10 seconds. The line `!msg.nil?` is the same as something like `msg != null` in Java code. In Ruby, however, even `null` is an object. Asking an object if it is of the `nil` type (via the `nil?` method call) returns a boolean.

In conclusion

Because AWS is a web services offering, it is accessed and leveraged by numerous platform libraries. In Magnus, you see the resulting flexibility: I was able to push messages onto an SQS queue using Java code, and then pop them off with a small Ruby program. One of the beauties of an architecture employing queues is that implicit decoupling of components.

Just as it often will make sense to host a web application on GAE or Amazon’s Elastic Beanstalk, it also will make sense to leverage a cloud messaging system. Amazon’s SQS elevates the burden of installing and maintaining a queuing system.

You simply create a queue, then drop and retrieve messages on it. Let Amazon worry about the rest.

Resources

Learn

- *Java development 2.0*: This dW series explores technologies that are redefining the Java development landscape. Topics include Amazon Elastic Beanstalk, Google App Engine (August 2009), and SimpleDB (June 2010).
- "Cloud computing with Amazon Web Services, Part 4: Reliable messaging with SQS" (Prabhakar Chaganti, IBM developerWorks, December 2008): More about Amazon Simple Queue Service (SQS).
- "Sacha Labourey on abstracting the cloud to work better for Java developers" (developerWorks, December 2010): The former JBoss CTO discusses his new endeavor, CloudBees, and argues that a complete set of tools for the application life cycle in the cloud (including Continuous Integration a la Hudson-as-a-service) is key to maximizing this new computing model — and besting GAE in the platform-as-a-service space.
- "Max Ross on the Google App Engine" (developerWorks, November 2010): Get details on the GAE platform, how it differs from EC2, how the datastore works, and where it is going in the future.
- Browse the Java technology bookstore for books on these and other technical topics.
- developerWorks Java technology zone: Find hundreds of articles about every aspect of Java programming.

Get products and technologies

- Amazon Simple Queue Service: Read the reference documentation and get started with Amazon SQS.
- The Play framework: Billed as a Java framework built by web developers, Play focuses on developer productivity and targets RESTful architectures.

Discuss

- Get involved in the developerWorks community. Connect with other developerWorks users while exploring the developer-driven blogs, forums, groups, and wikis.

About the author

Andrew Glover

Andrew Glover is a developer, author, speaker, and entrepreneur with a passion for behavior-driven development, Continuous Integration, and Agile software development. He is the founder of the easyb Behavior-Driven Development (BDD) framework and is the co-author of three books: Continuous Integration, Groovy in Action, and Java Testing Patterns. You can keep up with him at his blog and by following him on Twitter.

Trademarks

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.